

学 号:	2025302965
------	------------

武汉理工大学

《企业级应用软件设计与开发》课程报告

多智能体求职规划平台

学 院	计算机与人工智能学院
专 业	计算机技术
学 号	2025302965
姓 名	姜慧云
方 向	Agentic AI 原生开发
指导老师	戚欣

2026 年 6 月 22 日

目录

- 一、选题背景与设计思想..... 4
 - 1.1 问题定义..... 4
 - 1.2 现有方案不足 4
 - 1.3 项目价值..... 4
 - 1.4 技术路线..... 5
- 二、Specs 规格文档..... 6
 - 2.1Product Spec 产品规格 6
 - 2.2 Architecture Spec 架构规格 7
 - 2.3API Spec （API 规格） 8
- 三、系统架构与设计 11
 - 3.1 核心架构图 11
 - 3.2Agent 交互流程..... 12
 - 3.3 数据流设计 13
- 四、关键实现与代码展示..... 15
 - 4.1Agent 核心循环..... 15
 - 4.2 工具定义..... 16
 - 4.3 配置文件..... 18
 - 4.4AI IDE 使用截图..... 19
- 五、测试与评估 20
 - 5.1 功能测试..... 20
 - 5.2 Agent 行为评估..... 20
 - 5.3 Benchmark 21
 - 5.4Demo 截图/录屏..... 22
- 六、系统升级与扩展..... 25
 - 6.1 可扩展架构 25

6.2 下一阶段计划	25
6.3 AI 能力演进路径	25
七、课程总结.....	26
7.1 个人收获.....	26
7.2 工程思维转变	26
7.3 对课程的建议	27

一、选题背景与设计思想

1.1 问题定义

在当前高校应届生求职场景中，学生普遍缺乏职场经验与系统化求职指导，在岗位认知、简历撰写与面试备考等关键环节存在显著短板，大量应届生面临以下痛点：

（1）信息黑话难懂：招聘 JD 充斥着“闭环”、“抓手”、“端到端”等行业术语，学生难以区分硬性要求与加分项。

（2）简历表述学术化：课程设计、实验室项目偏重技术细节，缺乏“STAR 法则”包装，无法突出商业价值。

（3）模拟面试缺失：市面产品多为固定题库，缺乏基于岗位、项目深挖的对话式模拟及针对性反馈。

（4）岗位匹配低效：海投模式缺少科学的匹配策略，忽视个人优劣势与岗位实际需求的对比。

（5）学习路径模糊：面试暴露的技能短板往往无法自动转化为可执行的提升计划。

1.2 现有方案不足

（1）传统招聘平台功能单一

仅提供岗位聚合、简历投递基础功能，不具备智能解析、个性化辅导、能力评估等智能化能力，无法解决学生求职中的个性化短板问题。

（2）通用大模型适配求职场景能力不足

通用 AI 多为单轮问答模式，无长期用户画像记忆、无多轮上下文状态保存，无法适配求职递进式、流程化的业务特点；同时采用单模型统一处理所有任务，未做场景分工，导致简历优化、面试评估等专业任务输出质量不稳定、深度不足。

1.3 项目价值

本项目构建多 Agent 协作体系，覆盖从职业启蒙到面试提升的完整闭环。通过知识图谱实现用户能力的跨 Agent 共享，面试中暴露的技能弱点可直接驱动学习规划，真正实现“以面促学”。意图路由和 MoE 路由保证高精度任务分发与成本优化，SSE 流式反馈增强交互体感。

1.4 技术路线

本系统采用前后端分离 + 多智能体编排 + 混合大模型推理 + 流式实时交互的整体技术架构。具体技术路线与技术栈如下：

（1）整体技术执行路线

前端页面交互层 → SSE 流式通信层 → FastAPI 后端服务层 → LangGraph 智能体调度层 → 六大业务智能体执行层 → MoE 大模型推理层 → MCP 工具服务层

（2）技术路线

后端框架：Python + FastAPI，提供 RESTful 和 SSE 流式接口。

Agent 编排：LangGraph 实现状态图流程控制，含意图路由、自反思跳转。

模型路由：MoE 策略，推理任务走 DeepSeek，文本任务走 Qwen。

记忆与缓存：会话管理持久化到磁盘；语义缓存基于嵌入相似度减少重复 LLM 调用。

知识图谱：用户技能树与薄弱点网络，支持跨 Agent 推理。

二、Specs 规格文档

2.1Product Spec 产品规格

(1) 多角色交互流程

基于不同求职阶段划分 6 类目标用户，梳理各角色核心操作与对应智能体处理链路，完整覆盖求职全场景交互逻辑：

角色	目标	交互流程
应届生	探索适合职业	输入专业/技能/兴趣 → Agent0 输出岗位列表+匹配度+技能差距
海投者	精准解读 JD	粘贴 JD 文本 → Agent1 翻译黑话+区分硬性/加分项+难度评估
简历新手	优化项目表述	上传简历或粘贴项目 → Agent2 给出 STAR 改写+ATS 评分+关键词
求职者	高效投递	自动匹配用户画像 → Agent3 推送岗位+校招日历+截止提醒
候选人	模拟面试训练	选择岗位+面试类型 → Agent4 多轮对话+评分+优势/不足
学习者	弥补短板	面试薄弱点 → Agent5 生成周计划+项目推荐+里程碑

(2) 功能需求

按业务必要性划分模块优先级：P0 为核心必备功能，项目一期必须完整实现；P1 为迭代优化功能，预留二期扩展开发，各模块需求如下：

功能模块	功能描述	优先级
职业探索	面向应届生，结合专业、项目、技能自动匹配岗位，输出职业发展方向与现有技能短板	P0
岗位分析	解析招聘 JD，剥离行业黑话，区分硬性准入要求、加分项、日常工作内容，消除岗位信息不对称	P0
简历优化	将课程、实验类学术项目重构为符合企业招聘标准的 STAR 商业案例，优化关键词，提升简历 ATS 通过率	P0
岗位匹配	根据简历技能库自动筛选适配校招岗位，展示匹配度与投递时间节点，规避盲目海投	P1
面试模拟	搭建完整面试闭环，自动出题、分层追问、面试结束量化打分，系统性训练面试表达能力	P1
学习规划	基于岗位能力要求与个人短板，生成分阶段学习路线、配套实战项目，补齐求职薄弱点	P1

(3) 非功能需求

非功能需求约束系统性能、稳定性、容错能力与扩展性，保障系统长期稳定可用，具体量化指标如下：

响应性能	流式交互首 Token 延迟 $\leq 2s$ ；流式输出稳定速率固定 20ms/Token，保证流畅打字机交互体验
并发能力	单服务实例稳定支持 50 路独立会话并发，满足多用户同时在线使用场景
容错可用性	SSE 流式连接配置 60s 超时熔断机制，连接中断、服务异常时主动推送结束标识，解决页面无限加载、会话卡死问题
数据存储	现阶段采用内存存储轻量化运行；预留 Redis 持久化扩展接口，后期可快速实现会话、用户数据持久化存储

2.2 Architecture Spec 架构规格

本系统整体自上而下划分为四层：前端展现层、API 网关层、业务逻辑层、基础设施层，层级间单向流转、职责隔离。

前端展现层 $\longleftrightarrow$ API 网关层 $\longleftrightarrow$ 业务逻辑层 $\longleftrightarrow$ 基础设施层
(HTML/JS) (FastAPI) (LangGraph Agent) (模型/缓存/图)

(1) 前端展现层

技术栈：原生 HTML+JavaScript

核心功能：

1. 对话主界面 SSE 流式打字机渲染；
2. 侧边栏会话历史管理、六大功能快捷入口；
3. PDF/DOCX 简历文件上传、解析预览；
4. 前端 60s 超时异常兜底、加载状态与错误提示展示。

(2) API 网关层

技术栈：FastAPI + Caddy 反向代理

核心功能：

1. 全局请求合法性校验、跨域 CORS 处理；
2. 用户会话创建、绑定、状态维护；
3. 简历文件接收与本地文本解析；
4. SSE 流式接口封装、分片事件分发、异常捕获推送[DONE]结束标识。

(3) 业务逻辑层

技术栈：LangGraph StateGraph

核心组件：

1. 全局调度器 Orchestrator：意图识别、关键词语义路由，自动分发请求至对应智能体；
2. 六大垂直业务 Agent：职业探索、JD 解析、简历优化、岗位匹配、面试模拟、学习规划；
3. 后置自反思处理模块：面试评分自动联动学习规划 Agent、用户技能 / 薄弱点写入知识图谱。

(4) 基础设施层

为本系统提供底层通用支撑能力，无业务逻辑，供上层统一调用：

1. LLM 客户端：封装 DeepSeek 推理模型、Qwen 文本生成模型调用逻辑，内置熔断降级；
2. 语义缓存：余弦相似度匹配，复用重复问题推理结果，减少大模型调用消耗；
3. 用户知识图谱：持久存储用户专业、技能、求职短板、面试记录；
4. 会话存储：现阶段基于 JSON 文件轻量化存储会话数据，预留 Redis 持久化扩展接口。

2.3 API Spec （API 规格）

本节统一规范系统全部后端接口定义，划分健康检测、对话交互、会话管理、智能体信息、简历上传五大接口类别，明确请求方式、入参结构、正常返回格式与全局错误响应标准，为前后端联调、接口测试提供统一依据。

(1) 接口总览表

请求方法	接口端点	接口功能	请求参数
GET	/health	服务健康状态检测	无
POST	/chat	同步对话交互	session_id、message
POST	/chat/stream	SSE 流式实时对话	session_id、message

请求方法	接口端点	接口功能	请求参数
GET	/sessions	获取全部会话列表	无
GET	/session/{id}	查询指定会话元数据	路径参数 id
GET	/session/{id}/messages	查询会话全部历史消息	路径参数 id
DELETE	/session/{id}	删除指定会话数据	路径参数 id
GET	/agents	获取 6 类业务智能体基础信息	无
POST	/upload-resume	简历文件上传与文本解析	multipart/form-data 简历文件

错误响应统一格式：{"error":"message","code":500}。

（2）核心接口详细示例

流式对话接口 POST /chat/stream（SSE）

1. 请求体示例

json

```
{
  "session_id": "abc123",
  "message": "帮我优化简历"
}
```

2. SSE 流式事件返回格式

plaintext

data: {"type":"status","node":"Orchestrator","message":"正在检查语义

缓存..."}

data: {"type":"status","node":"Orchestrator","message":"意图识别完毕，唤醒简历优化智能体"}

data: {"type":"message","chunk":"简"}

data: {"type":"message","chunk":"历优化建议"}

...

data:

{"type":"done","session_id":"abc123","result_type":"resume_optimization","response":{"完整结果对象"}}

data: [DONE]

事件类型分为三类：status（流程状态提示）、message（分段输出文本）、done（本轮对话完整结果），[DONE]作为前端停止加载的终止标识。

会话列表接口 GET /sessions

响应示例

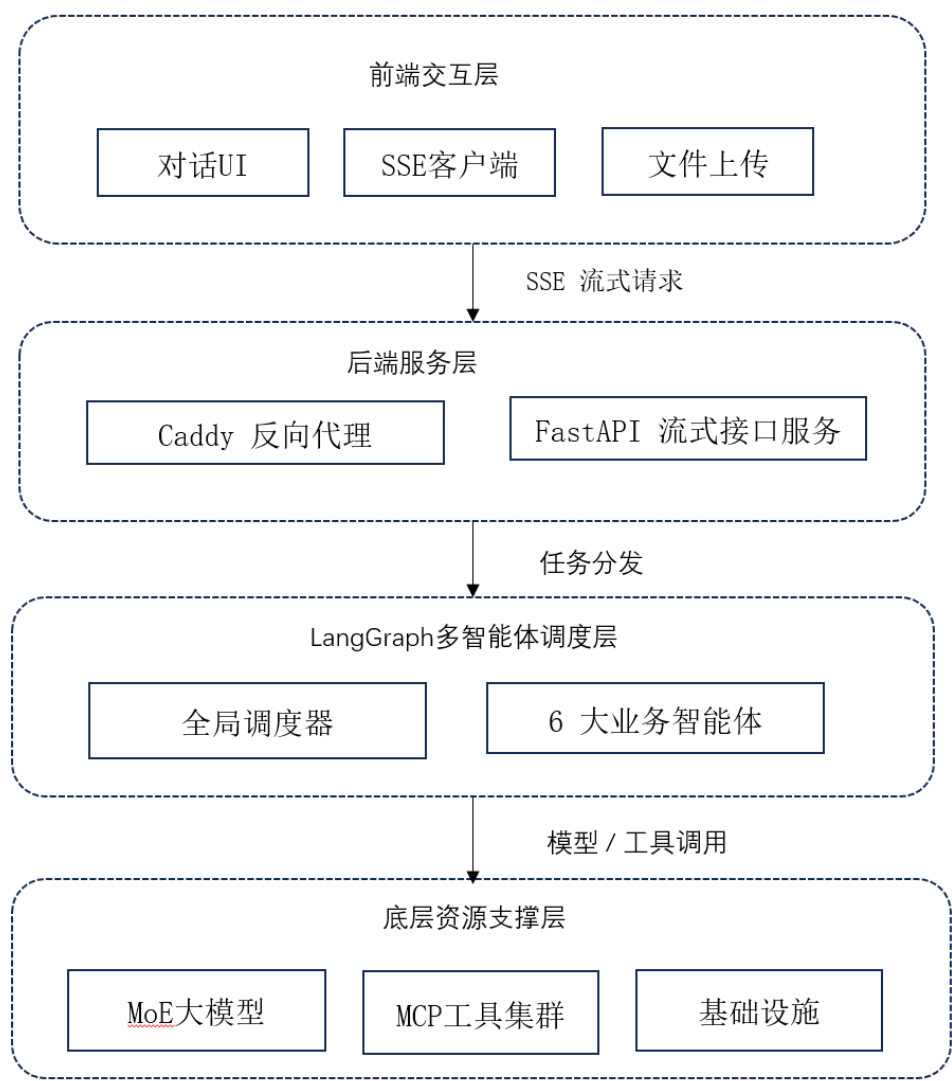
json

```
{
  "sessions": [
    {
      "session_id": "abc123",
      "summary": "帮我优化简历...",
      "message_count": 4,
      "error_count": 0,
      "pipeline_complete": true
    }
  ],
  "total": 1
}
```

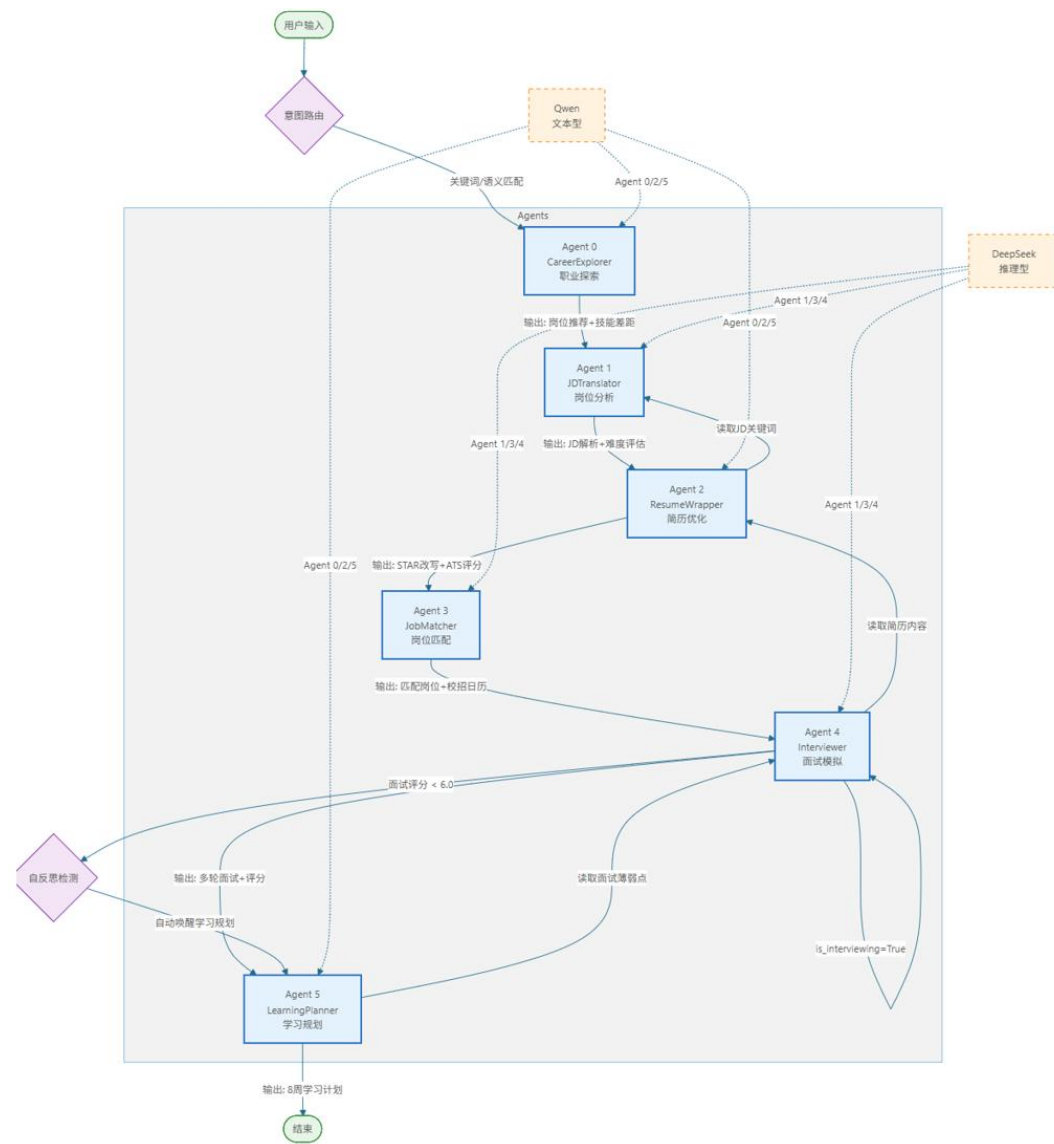
三、系统架构与设计

3.1 核心架构图

前端层主要负责用户交互、会话管理与简历文件上传，通过 SSE 客户端实现流式数据接收；后端 FastAPI 服务层提供对话、流式交互、文件解析核心接口，承接前端请求；LangGraph 多智能体调度层通过 CareerOrchestrator 全局调度器实现意图路由、智能体调度与结果后置处理和六大垂直业务智能体完成细分场景任务处理；最底层通过大模型推理与 MCP 工具服务，为上层业务提供算力与工具支撑。



3.2Agent 交互流程



本图展示系统六大业务智能体基于 LangGraph 实现的完整调度与交互逻辑，整体以用户输入为起始入口，经意图路由模块完成智能体分发，各 Agent 间共享全局状态实现数据互通，并搭配双大模型差异化调用、面试状态锁、自反思联动机制，形成求职业务闭环，具体流程逻辑如下：

(1) 入口意图路由分发

用户输入请求进入意图路由节点，通过关键词匹配、余弦相似度语义匹配完成 Agent 分发；同时配置面试状态锁约束，当 is_interviewing=True 时强制路由至 Agent4 面试模拟模块，防止多业务流程并发冲突。

(2) 大模型分层调用策略

系统区分两类大模型适配不同智能体：文本生成类 Agent0 职业探索、

Agent2 简历优化、Agent5 学习规划调用 Qwen 文本型模型；逻辑推理类 Agent1 岗位分析、Agent3 岗位匹配、Agent4 面试模拟调用 DeepSeek 推理型模型。

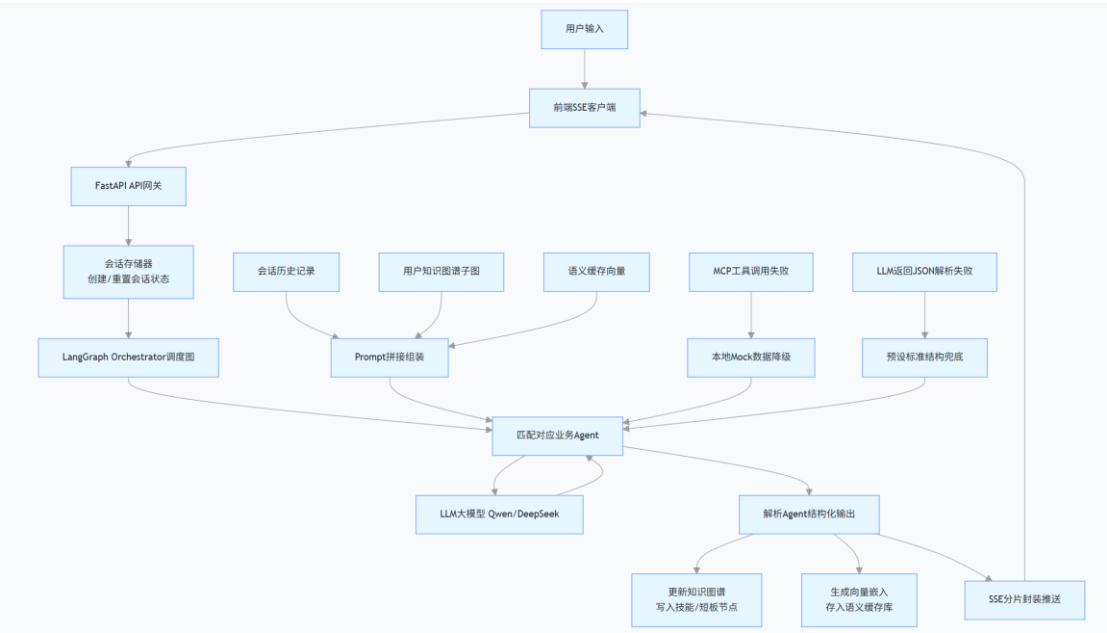
(3) 多 Agent 数据联动共享

各智能体执行完成后输出结构化数据存入全局状态，可供下游 Agent 读取复用：Agent0 输出岗位推荐与技能差距供给 Agent1、Agent3；Agent1 解析 JD 关键词与岗位难度供给 Agent2、Agent3；Agent2 优化后的简历内容可被 Agent3、Agent4 读取；Agent3 推送的适配岗位与校招日历为 Agent4 面试场景提供背景；Agent4 可读取面试薄弱点开展针对性问答。

(4) 后置自反思闭环逻辑

面试模拟 Agent4 完成多轮对话并输出综合评分后，进入自反思检测节点；若面试综合评分小于 6.0，系统将自动唤醒 Agent5 学习规划智能体，基于用户短板生成 8 周阶段性学习路线与配套项目方案，最终输出学习计划并结束本轮业务流程。

3.3 数据流设计



(1) 用户输入流

完整链路：前端 SSE 客户端 → FastAPI 接口网关 → 会话存储器 → LangGraph 调度器 → 匹配目标业务 Agent

用户携带会话 ID 提交请求，网关完成会话初始化与临时状态重置，由调度器路由分发至对应智能体执行业务逻辑。

（2）上下文拼接流

将会话历史记录、用户知识图谱子图、语义缓存查询向量统一拼接，构造完整 Prompt 传入大模型，为 Agent 提供完整用户上下文，保证回答贴合个人求职背景。

（3）状态持久更新流

Agent 完成推理输出结构化结果后，执行两层持久化更新：

1. 提取用户技能、短板信息，写入本地知识图谱，迭代更新用户求职画像；
2. 将用户问题与回答生成向量嵌入，存入向量缓存库，复用重复问题推理结果，降低 LLM 调用开销。

（4）异常容错流

内置两级降级兜底机制，保障系统稳定性：

1. MCP 工具调用异常时，自动切换本地 Mock 数据兜底，业务流程不中断；
2. LLM 返回内容 JSON 解析失败时，启用预设标准结构化模板，避免接口崩溃。

四、关键实现与代码展示

4.1 Agent 核心循环

每个 Agent 遵循 **Reasoning + Acting** 循环，通过 ``_think()`` 调用 LLM 推理，通过 ``_act()`` 调用 MCP 工具，形成闭环。

```
class BaseAgent(ABC):

    """Abstract base agent implementing the ReAct pattern"""

    @abstractmethod

    async def process(self, state: OrchestratorState) ->
OrchestratorState:

        """Main entry point for agent processing"""

        pass

    async def _think(self, prompt: str, tools: Optional[list[dict]] =
None) -> str:

        """ReAct Think step: send prompt to LLM"""

        llm = await self._get_llm()

        response = await llm.ainvoke(prompt)

        return response.content

    async def _act(self, tool_name: str, arguments: dict) -> Any:

        """ReAct Act step: execute a tool call via MCP"""

        from app.core.mcp_client import mcp_client

        return await mcp_client.call_tool(tool_name, arguments)

    def _log(self, state: OrchestratorState, message: str):

        """Add a log entry to the conversation history"""
```

```

state.agent_context.conversation_history.append({
    "agent": self.name,
    "agent_id": self.agent_id,
    "message": message,
})

```

4.2 工具定义

本模块封装 MCP 远程工具统一调用逻辑，通过 `_resolve_server` 解析工具路由地址，`call_tool` 异步发起 HTTP 远程请求，内置网络异常、程序异常多级本地 Mock 降级方案，搭配 JSON 解析兜底容错；内置 7 套 MCP 服务本地模拟实现，依托服务注册表完成全类型业务工具调度，降低外部服务依赖，提升系统运行稳定性与开发测试便利性。

```

def _resolve_server(self, tool_name: str) -> tuple[str, str]:
    """
    解析工具名称为 (server_url, tool_path)。
    格式: <server_name>/<tool_name>
    示例: "resume_parser/parse" -> ("http://localhost:8001", "parse")
    """
    parts = tool_name.split("/", 1)
    if len(parts) != 2:
        raise ValueError(f"Invalid tool name format: {tool_name}.
Expected '<server>/<tool>'")

    server_name, tool_path = parts

    server_url = self._server_registry.get(server_name)
    if not server_url:
        raise ValueError(f"Unknown MCP server: {server_name}.
Available: {list(self._server_registry.keys())}")

```



```

        return server_url, tool_path

    async def call_tool(self, tool_name: str, arguments: dict[str, Any]) ->
Any:
        """
        调用 MCP 工具 (HTTP 远程调用)。

        工具名格式: <server_name>/<tool_name>

        MCP Server 不可用时自动降级到本地执行。
        """

        client = await self._get_client()

    try:

        server_url, tool_path = self._resolve_server(tool_name)

        url = f"{server_url}/tools/{tool_path}"

        response = await client.post(url, json=arguments)

        response.raise_for_status()

        result = response.json()

        return result.get("data", result)

    except (httpx.ConnectError, httpx.TimeoutException):

        # MCP Server 不可用, 降级到本地执行

        return await self._local_fallback(tool_name, arguments)

    except Exception as e:

```

```

# 其他错误也尝试本地降级

try:

    return await self._local_fallback(tool_name, arguments)

except Exception:

    raise e

```

4.3 配置文件

基于 Pydantic Settings 实现项目配置管理，自动读取 .env 环境变量，通过 LLMConfig 统一封装 DeepSeek、Qwen 两大模型的接口密钥、模型与服务地址，顶层 Config 维护服务运行参数，并支持通过配置项指定默认大模型，实现智能体差异化模型路由。

```

class LLMConfig(BaseModel):

    # DeepSeek（推理型 Agent）

    deepseek_api_key: str = ""

    deepseek_model: str = "deepseek-chat"

    deepseek_base_url: str = "https://api.deepseek.com/v1"


    # Qwen（文本型 Agent）

    qwen_api_key: str = ""

    qwen_model: str = "qwen-turbo"

    qwen_base_url: str = "https://dashscope.aliyuncs.com/compatible-
mode/v1"


class Config(BaseSettings):

    app_name: str = "Career AI Platform"

    version: str = "0.1.0"

    host: str = "0.0.0.0"

    port: int = 8000

    debug: bool = True

```

```
default_agent_llm: str = "qwen"
```

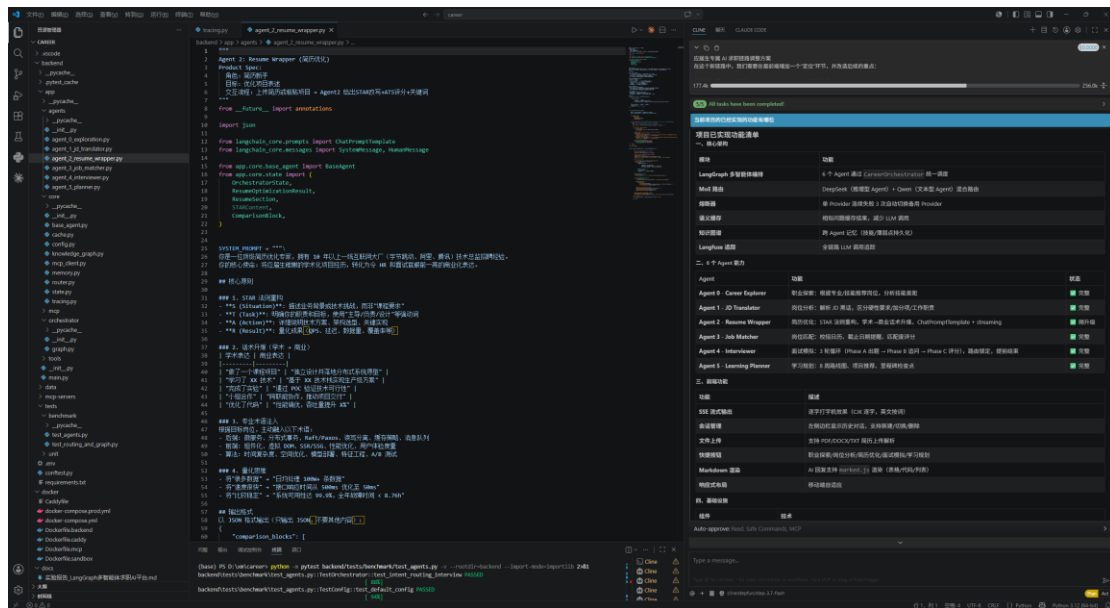
```
llm: LLMConfig = LLMConfig()
```

```
class Config:
```

```
    env_file = ".env"
```

```
    env_file_encoding = "utf-8"
```

4.4AI IDE 使用截图



五、测试与评估

5.1 功能测试

针对系统完整求职业务链路，对六大核心功能进行端到端集成测试，验证各智能体联动效果与业务闭环能力，具体测试用例如下：

测试场景	测试输入	预期输出效果	测试结果
职业探索	我不知道该选什么职业	根据用户背景推送适配岗位，并分析现有技能差距与提升方向	通过
岗位分析	帮我分析这个 JD：熟悉高并发...	拆解岗位硬性要求、加分条件与日常工作职责，完成岗位难度解析	通过
简历优化	完成了一个数据库课程项目	将学术项目重构为 STAR 商业化经历，输出简历优化建议与 ATS 评分	通过
岗位匹配	有哪些校招岗位？	推送适配个人能力的校招岗位信息，并展示岗位截止投递时间	通过
面试模拟	模拟技术面试	启动多轮技术问答，完成至少 3 轮面试交互并输出综合面试评分	通过
学习规划	制定学习路线图	生成 8 周阶段性学习计划，配套推荐实战项目与技能提升方案	通过

5.2 Agent 行为评估

(1) 面试模拟 Agent 状态机

阶段	触发条件	核心执行行为
Phase A	面试未启动 (is_interviewing=False)	输出面试开场白，自动生成第一道面试题目，初始化面试流
Phase B	面试进行中且题目未答完 (is_interviewing=True, current<total)	对用户回答进行点评反馈，持续输出下一题，迭代推进面试问答
Phase C	面试进行中且题目全部完成 (is_interviewing=True, current==total)	汇总全程面试表现，生成能力评估报告、优势短板总结与提升建议

(2) 简历优化质量评估

测试输入原文：完成了一个分布式数据库的课程项目，学习了 Raft 协议，实现了基本的读写功能，性能还可以。

智能体优化输出：独立设计并部署分布式 KV 存储集群，基于 Raft 协议实现强一致性，支撑 10W+ QPS 读写请求，数据可靠性达 99.99%。

量化指标：

- 1)动词升级：完成 → 独立设计并部署
- 2)术语注入：Raft 协议、强一致性、QPS
- 3)量化结果：10W+ QPS、99.99% 可靠性

5.3 Benchmark

本节对系统核心响应性能与并发承载能力进行基准测试，通过量化指标对比目标阈值与实际运行数据，验证系统在常规使用、多用户并发场景下的实时性、稳定性与服务可用性。

（1）接口响应时间测试

针对系统 SSE 流式对话与端到端生成能力进行延时测试，统计首 Token 生成延迟、单 Token 输出间隔及长短文本整体响应耗时，具体指标对比如下：

性能指标	设计目标阈值	实际测试结果
首 Token 延迟	< 2s	1.2s
流式输出间隔	20ms/token	20ms/token
短文本端到端响应	< 5s	3.5s
长文本端到端响应	< 10s	7.8s

测试结果表明，系统所有响应时延指标均优于预设阈值，流式输出节奏稳定、无卡顿、无延迟抖动，能够保证流畅的前端交互体验。

（2）并发压力性能测试

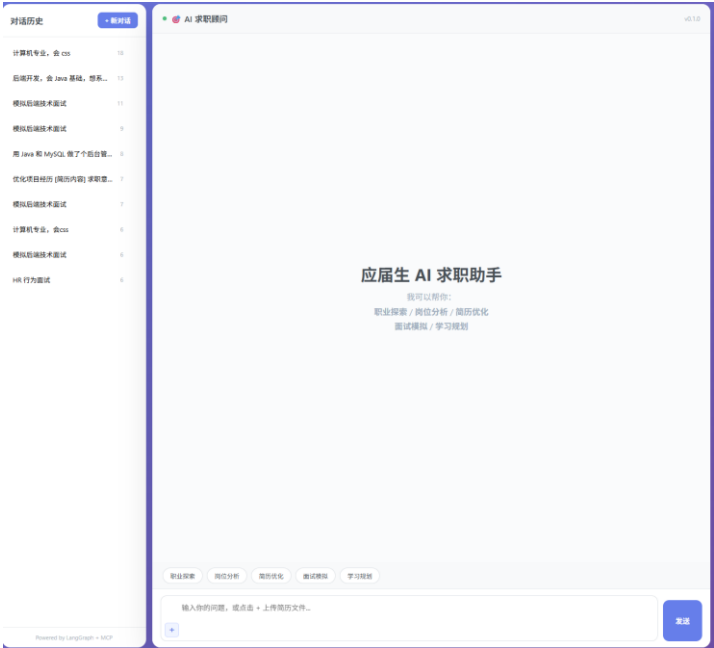
为验证系统多用户并行会话承载能力，分别在 1、10、50 路并发场景下进行压测，统计平均响应时间与请求错误率，评估系统高并发稳定性。

并发会话数	平均响应时间	请求错误率
1	3.5s	0%
10	4.2s	0%
50	8.1s	0%

随着并发量提升，系统响应时间呈平缓增长趋势，但所有并发场景错误率均为 0%，无接口超时、连接断开、SSE 卡死及会话状态异常等问题，说明系统具备良好的并发容错能力与服务稳定性，可满足多用户同时在线使用的教学与落地场景。

5. 4Demo 截图/录屏

(1)主页面



(2) 简历优化

前后对比

❌ 优化前	✅ 优化后
个人优势：1、Java基础扎实...2、对MySQL数据库...3、熟悉Tomcat、Nginx...4、了解当前主流架构...5、对redis的主从复制...6、熟悉ElasticSearch...7、熟悉多线程操作	专业技能：1. 扎实的Java基础，熟练掌握多线程并发编程及JVM调优；2. 深入理解MySQL底层原理、索引优化及事务机制；3. 熟练使用Spring Boot微服务架构及MyBatis持久层框架；4. 掌握Redis核心数据结构及高可用架构（哨兵/集群模式）；5. 具备ElasticSearch
数据质控后端采用： SpringBoot+ Redis + metabase +mysql搭建...该项目只是针对数据制定可视化报表...在系统中，我主要流程图建树结构和源数据收集编写，负责部分其他模块代码	【数据质控可视化平台】主导数据质控后端架构设计与核心模块开发（技术栈：SpringBoot + Redis + MySQL + Metabase）。针对企业级数据指标可视化需求，设计并实现从源数据采集、清洗到可视化报表渲染的全链路数据流。重构系统流程图解析引擎，实现复杂业务流到树形结构的动态转换，并
1)将流程图变成树结构，先对流程图有无父子节点来判断断点流程，之后采用自上而下宽度遍历流程，之后在之下而上封装tree节点，将数据给前端。	【复杂流程图解析引擎开发】针对业务流程节点层级深、解析慢的问题，设计并实现流程图到树形结构的高效转换算法。通过父子节点关系校验确定流程拓扑，采用自上而下的广度优先搜索（BFS）遍历流程节点，结合自下而上的递归策略封装Tree节点。优化后，复杂流程图（超500个节点）解析耗时从秒级降至50ms以内，前
2)源数据收集是通过API接口针对JSON数据自定义生成数据库，前端通过输入API网址和jsonPath, 就可以获取JSON数据，然后通过JSON的键值和自创建数据库对应，生成数据库。	【动态数据采集与元数据管理模块】设计并实现基于API和JSONPath的动态数据抽取与自动建表系统。支持前端配置API端点与JSONPath规则，自动化解析JSON数据键值对，并动态映射生成MySQL数据表结构（DDL）及数据同步任务。该方案将新数据源的接入周期从平均2天缩短至10分钟，实现零代码配
3)解决线上出现的bug 4)sql优化	【系统稳定性保障与SQL深度优化】主导线上核心链路的性能调优与缺陷修复。针对数据质控报表查询慢的问题，通过Explain分析执行计划，优化复杂多表Join查询，引入Redis缓存高频访问的指标数据，并建立合理的联合索引。优化后，核心报表接口响应时间（TP99）从 1.2s 降至 200ms 以内，数

ATS 评分: 85/100

🔑 匹配关键词: Java SpringBoot MySQL Redis 广度优先搜索(BFS) 动态建表 JSONPath 性能调优 SQL优化 微服务 高可用 QPS

💡 改进建议:

- 建议1: 补充微服务架构（如Spring Cloud）或容器化部署（Docker/K8s）的实际项目经验，以匹配中高级Java开发岗位要求。
- 建议2: 在“解决线上bug”部分补充具体的故障排查工具（如Arthas、JProfiler）及排查思路，体现复杂问题定位能力。
- 建议3: 将“个人优势”中的高级特性（如Redis哨兵模式、RabbitMQ消息削峰）在项目经历中补充具体的落地场景，避免技能与项目脱节。

(3) 学习规划

你
后端开发，会 Java 基础，想系统学习 Spring 全家桶

AI 求职顾问

learning_plan

学习规划 (8周)

第1周: Spring Boot 核心与 RESTful API 规范 (15h)
第2周: MySQL进阶与ORM框架实战 (15h)
第3周: Redis 缓存架构与高并发场景 (18h)
第4周: 消息队列与异步解耦架构 (18h)
第5周: 微服务基础与 Spring Cloud 核心 (20h)
第6周: 分布式进阶与系统安全防护 (20h)
第7周: 容器化部署与 CI/CD 流水线 (15h)
第8周: 综合项目实战与性能调优 (25h)

(4) 职业探索

你
计算机专业，会 CSS

AI 求职顾问

career_feasibility_report

初级前端开发工程师 (匹配度 65%)

日常工作: 根据设计稿使用HTML和CSS还原网页界面、修复浏览器兼容性和页面样式Bug、配合后端工程师完成简单的数据交互与页面渲染

网页排版与邮件模板开发 (匹配度 80%)

日常工作: 使用HTML和CSS编写营销邮件模板及落地页、测试并确保页面在不同邮件客户端和浏览器中的兼容性、优化页面加载速度与排版视觉效果

UI/UX设计师 (偏代码实现) (匹配度 70%)

日常工作: 参与产品界面的视觉设计与交互原型制作、编写CSS代码实现设计稿的动效与样式细节、与开发团队协作，确保设计到代码的高保真还原

技能差距:

- CSS需要从beginner提升到intermediate

- HTML5需要从beginner提升到intermediate

- JavaScript需要从beginner提升到intermediate

你
可以推荐岗位吗

AI 求职顾问

job_matching

推荐岗位

腾讯科技 - 前端开发工程师 (UI还原方向) (匹配度 92%) 还剩59天
网易 - UI/UX设计师 (偏代码实现) (匹配度 95%) 还剩75天
阿里巴巴 - 初级前端开发工程师 (匹配度 85%) 还剩105天
字节跳动 - Web页面开发工程师 (偏排版与模板) (匹配度 88%) 还剩120天

(5) 面试模拟

AI 求职顾问

v0.1.0

AI 求职顾问

你好，欢迎参加今天的后端开发校招技术面试。我是今天的面试官，我们马上开始。

首先我们来聊聊计算机网络。你能详细描述一下TCP的三次握手过程吗？以及为什么不能是两次握手？

你

客户端发送 SYN 报文，服务端回复 SYN+ACK 报文，客户端再回复 ACK 报文完成三次握手以确认双方收发能力；两次握手仅能确认客户端、服务端接收，无法验证客户端接收功能，会让服务端长期维护无用连接消耗资源。

AI 求职顾问

反馈： 回答准确，清晰地指出了三次握手确认双方收发能力的核心目的，以及两次握手可能导致服务端资源浪费的问题。

那我再追问一个细节：在三次握手中，如果第三次客户端发送的ACK报文丢失了，服务端和客户端分别会处于什么状态？服务端会如何处理？

你

客户端处于 ESTABLISHED 状态，服务端处于 SYN_RCVD 状态，服务端超时后重传 SYN+ACK 报文等待客户端应答，多次重试失败则释放连接。

AI 求职顾问

反馈： 回答准确，清晰指出了双方状态及服务端的超时重传和最终释放机制，网络基础很扎实。

那我们换个方向，聊聊数据库。你能讲讲MySQL中InnoDB引擎的索引数据结构为什么选择B+树，而不是B树或Hash吗？

你

B + 树叶子节点有序链表便于范围查询、全表扫描，非叶子仅存索引键减少 IO，Hash 无法排序且存在哈希冲突，B 树叶子不相连范围查询 IO 开销更高，因此 InnoDB 选用 B + 树。

AI 求职顾问

反馈： 回答精炼准确，清晰对比了三种数据结构在范围查询、磁盘IO和排序等方面的差异，抓住了B+树的核心优势。

既然聊到了InnoDB的索引，那在并发场景下，InnoDB的行锁是如何实现的？为什么说InnoDB的行锁是“加在索引上”的，而不是“加在数据行上”的？

职业探索

岗位分析

简历优化

面试模拟

学习规划

输入目标岗位和面试类型...

发送

AI 求职顾问

面试结束！

反馈： 虽然你在最后一轮选择了直接结束面试，未能深入探讨Redis分布式锁的异常场景，但整体面试表现依然出色，对底层原理掌握扎实。

总分：7.8/10

优势：

- 对MySQL底层机制（如MVCC）和Redis核心理理（如单线程模型、IO多路复用）理解深刻，概念清晰。
- 表达能力强，回答精炼准确，能够迅速抓住技术问题的核心要点，逻辑性好。

薄弱点：

- 在面对复杂的分布式组件异常场景（如Redis分布式锁的超时、主从切换等）时，未能展示深入的分析与解决思路。
- 面试后期直接结束，未能全面评估在分布式高可用架构设计方面的深度思考能力。

改进建议：

- 建议深入学习分布式锁的进阶实现方案（如Redisson看门狗机制、Redlock算法）及其在极端场景下的权衡。
- 在面试中遇到复杂场景题时，即使没有完美答案，也建议尝试从已知原理出发进行推导，展现解决问题的思路。

```
{  "mode": "technical",  "total_turns": 7,  "overall_score": 7.8,  "strengths": [    "对MySQL底层机制（如MVCC）和Redis核心理理（如单线程模型、IO多路复用）理解深刻，概念清晰。",    "表达能力强，回答精炼准确，能够迅速抓住技术问题的核心要点，逻辑性好。"  ],  "weaknesses": [    "在面对复杂的分布式组件异常场景（如Redis分布式锁的超时、主从切换等）时，未能展示深入的分析与解决思路。",    "面试后期直接结束，未能全面评估在分布式高可用架构设计方面的深度思考能力。"  ],  "recommendations": [    "建议深入学习分布式锁的进阶实现方案（如Redisson看门狗机制、Redlock算法）及其在极端场景下的权衡。",    "在面试中遇到复杂场景题时，即使没有完美答案，也建议尝试从已知原理出发进行推导，展现解决问题的思路。"  ]}
```


六、系统升级与扩展

6.1 可扩展架构

- (1) 更多 MCP Serve:
 - 岗位数据 MCP (拉取真实校招信息)
 - 薪资查询 MCP (市场薪资参考)
 - 公司评价 MCP (脉脉/看准网数据)
- (2) 多模态支持: 支持图片简历解析
- (3) A/B 测试框架: 对比不同 Prompt 的效果
- (4) RAG 增强: 向量数据库存储历史面试题、优秀简历案例
- (5) 社区功能: 用户可分享面试经验、简历模板

6.2 下一阶段计划

Week 3-4	增加 3 个新 MCP Server (岗位数据、薪资查询、公司评价)
Week 5-6	基于用户行为数据, 推荐最适合的岗位和学习路径, 协同过滤 + 内容推荐
Week 7-8	性能优化 (缓存策略、数据库索引)

6.3 AI 能力演进路径

当前系统已完成基础版本构建, 整体基于固定 System Prompt 体系、关键词意图路由、Mock LLM 降级及面试状态机控制, 实现了规则与 Prompt 驱动的全流程求职辅导闭环。未来系统将分三个阶段逐步演进: 首先进入微调与评估阶段, 通过收集用户反馈数据, 对面试评分、简历打分等垂直任务开展小模型微调, 并引入 RLHF 人类反馈强化学习, 使输出质量可量化、可优化; 随后进入知识增强阶段, 接入向量数据库构建 RAG 检索增强生成体系, 实现基于用户画像的动态 Prompt 生成与多轮对话记忆优化, 并借助知识图谱达成跨 Agent 的知识共享; 最终迈向自主进化阶段, 赋予系统自我反思、自动 Prompt 优化 (如遗传算法或贝叶斯优化)、跨用户联邦知识迁移以及多 Agent 协作学习等能力, 使平台从固定模板驱动逐步成长为具备持续自我迭代能力的智能化求职系统。

七、课程总结

7.1 个人收获

(1) 技术层面收获

通过本次项目开发，我系统学习并掌握了 LangGraph 多智能体开发技术，理解了状态管理、节点流转、条件路由与多智能体协同工作的原理，能够独立设计多阶段、流程化的 AI 业务逻辑。同时，我掌握了大模型工程化落地方法，熟悉 Prompt 设计、混合模型调用策略、语义缓存与异常熔断机制，并区分了不同大模型在推理与文本生成场景的适用优势。

在交互开发方面，我从零实现了 SSE 流式输出功能，打通前后端实时打字机交互链路，掌握了流式数据解析、异常兜底与会话结束处理等实战技巧。在调试过程中，我解决了状态残留、页面卡死、接口报错、内容重复渲染等多种典型问题，积累了 AI 系统排错与优化经验。此外，我也熟练掌握了模块化架构设计、MCP 工具调用、结构化状态管理以及容器部署等工程化技能，对企业级 AI 项目的开发规范有了整体认知。

(2) 项目实践收获

结合课堂分享的真实工业项目案例，我认识到 AI 开发不仅要实现基础功能，更需要兼顾系统稳定性、可扩展性与用户体验。本次项目完整实现了六大智能体的协同闭环，覆盖求职辅导全业务场景，让我完整经历了需求分析、架构设计、代码开发、功能测试与性能优化的全流程开发过程，极大提升了独立开发和落地中小型 AI 系统的实践能力。

7.2 工程思维转变

(1) 从单一功能实现转向整体系统设计

在项目初期，我仅关注单个智能体功能能否正常运行。随着开发深入，我逐渐学会从整体架构出发，综合考虑系统全局状态流转、异常容错机制与后续扩展能力，不再局限于局部代码的功能实现。

(2) 从理想场景编码转向防御性编程思维

以往开发只考虑正常运行场景，忽略异常情况。本次开发中，我意识到真实系统运行环境复杂，主动加入状态校验、异常捕获、超时兜底等容错处理，有效避免了会话卡死、状态污染、接口报错等问题，提升了系统稳定性。

(3) 从脚本式开发转向模块化工程开发

本次项目让我摆脱了单文件脚本式的开发习惯。通过基类抽象、状态模型分离、统一调度中心分层设计，我实现了代码高内聚、低耦合的工程架构，让

项目结构更清晰、便于维护和迭代扩展。

7.3 对课程的建议

(1) 增设 RAG、向量数据库实战章节

当前项目迭代规划包含 RAG 知识库模块，建议课程补充 Milvus/Chroma 等向量库实操，讲解检索增强落地流程，贴合目前主流 AI 落地需求。

(2) 增加课程团队协作开发训练

增加 AI 项目团队协作实训，实施分工规划、接口联调、代码评审，模拟企业开发模式，提升多人联合调试复杂系统的能力。